

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART 21

Competitive Programming Patterns

How to read an unfamiliar problem, extract its constraints, and match it to a technique — turning everything from Parts 1-19 into a single reusable decision process

PART 21

Competitive Programming Patterns

Not a new topic — the index to everything you already know

Where this Part fits

This Part corresponds to the roadmap's Part XXII (“Competitive Programming Patterns”) and Phase 21 (“Competitive Programming”), delivered here as Part 21. It assumes everything from Parts 1–19 and doesn't introduce new data structures or algorithms — instead it's the connective layer: given a fresh problem statement, how do you decide WHICH of those tools applies?

Every technique in this book solves a category of problem. The hardest part of competitive programming is rarely implementing Dijkstra or a segment tree once you know that's what's needed — it's recognizing, from a problem statement and its constraints, that Dijkstra or a segment tree is the tool at all. This Part is entirely about that recognition step.

Contents of Part 21

- 1. The Pattern-Recognition Habit
- 2. Constraints → Algorithm Selection
- 3. Which Data Structure Should I Use?
- 4. Which Algorithm Should I Use?
- 5. Pattern Catalog: Clue Words → Technique
- 6. Worked Walkthrough: From Statement to Solution
- Part 21 Revision, Cheat Sheet, and Interview Questions

Chapter 1 — The Pattern-Recognition Habit

1.1 Why this is a separate skill from knowing algorithms

Knowing how binary search works and recognizing that a problem needs binary search are different skills, trained differently. The first comes from reading Part VI once. The second comes from deliberately asking the same short list of questions every time, before writing any code, until it becomes automatic.

1.2 The process, in order

Step	Question to ask	What it narrows down
1. Understand	What exactly is the input, and what exactly is the output?	Rules out misreading the problem entirely
2. Constraints	How large can n get? What's the time limit?	Tells you the TARGET complexity (Chapter 2)
3. Examples	Do the sample input/output make sense to me by hand?	Catches misunderstandings before coding
4. Brute force	What's the obvious, definitely-correct slow approach?	A fallback, and a source of insight into the structure
5. Bottleneck	Which specific part of the brute force is slow, and why?	Points directly at what needs to change
6. Pattern match	Which of the standard patterns (Chapter 5) does the bottleneck resemble?	Suggests the actual technique to apply
7. Verify	Does the chosen technique's complexity actually fit the constraints from step 2?	Confirms the choice before investing time coding it

✓ **This is Part II's framework, repeated because it's the whole point**

Part II introduced “How to Think Before Coding” as a general habit. This Part is that same process, but with Chapters 2–5 supplying the actual lookup tables you consult at steps 2 and 6 — so the process stops being abstract advice and becomes something you can literally run through on a real problem.

1.3 A worked example of just the recognition step

Problem: “Given an array of $n \leq 200,000$ integers, answer $q \leq 200,000$ queries: does any subarray of a given range sum to exactly zero?”

Observation	Conclusion
n, q both up to 2×10^5	need roughly $O((n+q) \log n)$ or better — $O(n^2)$ (10^{11}) is far too slow
"subarray sum" is the key phrase	prefix sums (Chapter 5) are almost always involved when a problem says 'subarray sum'
"sum to exactly zero" over a RANGE, repeated queries	suggests a range-query structure over prefix sums — points toward Fenwick/segment tree or offline processing

Notice that no code was written yet — this entire table is the recognition step, and it already narrows “some data structure problem” down to “prefix sums plus a range-query structure,” before a single line is typed.

 Quick Revision

- Recognizing which technique applies is a distinct, trainable skill from knowing how the technique works.
- Always read constraints before choosing an approach — they tell you the target complexity, which rules out most wrong turns immediately.
- Bottleneck-first thinking (find the slow part of the brute force, then ask what removes it) is more reliable than pattern-matching the whole problem to a memorized template.

Chapter 2 — Constraints → Algorithm Selection

2.1 The core table

Competitive judges typically allow roughly 10^8 – 10^9 simple operations per second. Working backward from n tells you the complexity class the intended solution almost certainly belongs to.

Constraint on n	Likely intended complexity	Typical techniques
$n \leq 10$	$O(n!)$ or exhaustive search	brute-force permutations, full search
$n \leq 20$	$O(2^n)$	bitmask enumeration, bitmask DP (Part 19)
$n \leq 40$	$O(2^{n/2})$	meet in the middle
$n \leq 100$	$O(n^3)$	Floyd-Warshall, simple 3D DP
$n \leq 1,000$	$O(n^2)$	pairwise checks, $O(n^2)$ DP
$n \leq 100,000$	$O(n \log n)$	sorting, binary search, segment/Fenwick tree, heaps
$n \leq 1,000,000$	$O(n)$ or $O(n \log n)$	linear scans, two pointers, sieve, prefix sums
$n \leq 10^9$ or higher	$O(\log n)$ or $O(1)$	binary search on the answer, math formulas, matrix exponentiation

⚠ This is a guideline, not a law

A tight constant factor can make $O(n^2)$ fail even for $n=2,000$ on some judges, and a loose one can let $O(n^2)$ pass for $n=5,000$. Always sanity-check against the SPECIFIC time limit and operation cost stated, especially when a problem's constraints sit right at a boundary between two rows of this table.

2.2 Reading multiple constraints together

Many problems give more than one bound — array size, value range, number of queries — and the intended complexity has to satisfy all of them simultaneously.

Given	What it suggests
$n \leq 10^5$, $q \leq 10^5$ (many queries on one array)	preprocess once in $O(n \log n)$, answer each query in $O(\log n)$ or $O(1)$ — $O(q \times n)$ per-query rescans are too slow
$n \leq 15$ (very small), but values up to 10^9	the SIZE (not the value range) signals $O(2^n)$ — bitmask DP, not something tied to the value magnitude
$n \leq 10^6$, values up to 10^9 , need range sums	value range is irrelevant to complexity here — use long long to avoid overflow (Part I), and a Fenwick/prefix-sum approach for the size

Quick Revision

- Work backward from n and the time limit to a target complexity BEFORE choosing a technique — it eliminates most wrong approaches immediately.
- Distinguish which given number actually drives complexity (often array size, not value range) — conflating them is a common misdiagnosis.
- Treat boundary cases (n right at 1,000 or 100,000) with extra caution; the table is a strong prior, not a guarantee.

Chapter 3 — Which Data Structure Should I Use?

Need	Structure	Part
Fast membership test (is x present?)	<code>unordered_set</code> / <code>set</code>	Part IV, VII
Key-value mapping	<code>unordered_map</code> / <code>map</code>	Part IV, VII
Sorted keys, need order statistics	<code>map</code> / <code>set</code> (ordered)	Part IV
Repeatedly need the min/max	<code>priority_queue</code>	Part XII
LIFO behaviour (most-recent-first)	<code>stack</code>	Part IX
FIFO behaviour (arrival order)	<code>queue</code>	Part IX
Need fast operations at BOTH ends	<code>deque</code>	Part IX
Next/previous greater or smaller element	monotonic stack	Part IX
Sliding-window maximum/minimum	monotonic deque	Part IX
Prefix search / autocomplete-style queries	trie	Part XVII
Connectivity / “same group?” queries	DSU (Disjoint Set Union)	Part XIII
Range sum with point updates	Fenwick Tree	Part XVIII
Range min/max, or range updates too	segment tree (+ lazy propagation)	Part XVIII
Static array, many range min/max/gcd queries	sparse table	Part XVIII
Repeated k-th ancestor / LCA queries on a tree	binary lifting	Part XVIII
Which subset of a small set has been used	bitmask (as a DP index)	Part 19

✓ When two structures both seem to fit

Prefer the simpler one first. A plain sorted vector with binary search often replaces a set/map when there are no interleaved insertions; a Fenwick Tree beats a segment tree whenever the query is plain range-sum with point updates (Part XVIII, Chapter 1). Reach for the more powerful structure only once the simpler one provably can't do the job.

Practice — Chapter 3

Problem 3.1 — Structure choice drill [Level 1]

For each situation, name the single best-fit structure from the table above: (a) count distinct elements seen so far while streaming input; (b) repeatedly report the two largest values in a changing collection; (c) check if two nodes in a growing graph are already connected as edges are added one at a time.

Hint 1: Match each situation's VERB (count distinct, report largest, check connected) to the 'Need' column above.

Approach: (a) `unordered_set`. (b) `priority_queue` (or two of them / a multiset). (c) DSU.

Complexity: Recognition itself is $O(1)$ once the pattern is familiar — that's the entire point of this chapter.

Chapter 4 — Which Algorithm Should I Use?

Situation	Algorithm	Part
Data is sorted, need to find/check a value	binary search	Part VI
Need the smallest value satisfying a monotonic condition	binary search on the answer	Part VI
Subarray sum queries, static array	prefix sums	Part III
Contiguous subarray satisfying a size/sum condition	sliding window	Part III
Pair/triplet search in sorted data	two pointers	Part III
Shortest path, unweighted graph	BFS	Part XIII
Shortest path, non-negative weights	Dijkstra	Part XIII
Shortest path, negative weights allowed	Bellman-Ford	Part XIII
All-pairs shortest paths, small graph	Floyd-Warshall	Part XIII
Minimum spanning tree	Kruskal or Prim	Part XIII
Dependency ordering (must happen before)	topological sort	Part XIII
Merging groups / connectivity queries	DSU	Part XIII
Local choice looks optimal at every step	investigate greedy (prove an exchange argument)	Part XIV
Overlapping subproblems + optimal substructure	dynamic programming	Part XV
Which subset of a small set has been used	bitmask DP	Part 19

Situation	Algorithm	Part
$n \leq \sim 20$, need to explore all subsets	bitmask enumeration	Part 19
$n \leq \sim 40$, split into two halves	meet in the middle	Part XIX
Exact pattern matching in a string	KMP or Z-algorithm	Part XVII
Many patterns, one text	Aho-Corasick	Part XVII

 **This table is a starting point for investigation, not a lookup you trust blindly**

“Local choice looks optimal” is deliberately phrased as “investigate greedy,” not “use greedy” — Part XIV's whole point is that greedy needs a correctness argument (an exchange argument, or a matching structure) before you trust it. Every row here is a strong hint, not a substitute for actually reasoning about the specific problem.

Practice — Chapter 4

Problem 4.1 — Algorithm choice drill [Level 1]

For each situation, name the algorithm: (a) find the minimum number of coins to make change, coin values not necessarily 'nice'; (b) determine build order given module dependencies; (c) find the shortest route on a road network with all-positive travel times.

Hint 1: (a) is a classic trap — greedy fails for arbitrary coin systems; check the 'when greedy fails' material in Part XIV.

Approach: (a) dynamic programming (unbounded knapsack style) — NOT greedy, unless the coin system is specifically canonical. (b) topological sort. (c) Dijkstra.

Complexity: Recognition is $O(1)$ once trained; correctness reasoning (especially for (a)) is the part that actually takes work.

Chapter 5 — Pattern Catalog: Clue Words → Technique

For each pattern: what to look for, what it typically costs, and the single most common mistake made when applying it.

Pattern	Clue words	Typical complexity
Frequency map	"count occurrences", "most/least frequent"	$O(n)$
Prefix sum	"subarray sum", "range sum query" (static)	$O(n)$ build, $O(1)$ query
Two pointers	sorted array, "pair/triplet summing to X"	$O(n)$ or $O(n \log n)$ with a sort
Sliding window	"contiguous subarray", fixed or variable size condition	$O(n)$
Binary search	sorted data, or "minimum X such that condition holds"	$O(\log n)$
Greedy	"maximize/minimize", local exchange feels safe	$O(n \log n)$ (often a sort first)

Pattern	Clue words	Typical complexity
BFS	unweighted shortest path, "minimum steps/moves", level-by-level	$O(V+E)$
DFS	explore all paths, connectivity, cycle detection	$O(V+E)$
Shortest path (weighted)	"minimum cost/time/distance" with weighted edges	$O((V+E) \log V)$ Dijkstra
DSU	"same group", "merge", "connected" queries over time	$O(\alpha(n)) \approx O(1)$ per op
Topological sort	"must come before", "build order", "course prerequisites"	$O(V+E)$
Heap / priority queue	repeatedly need current min/max, "k-th largest/smallest"	$O(n \log n)$

Pattern	Clue words	Typical complexity
Monotonic stack	"next/previous greater/smaller element"	$O(n)$ amortized
Interval problems	"overlapping intervals", "merge ranges", scheduling	$O(n \log n)$ (sort by start/end)
Recursion	problem defined in terms of a smaller version of itself	varies — see Part X
Backtracking	"all possible", "every arrangement/combination"	exponential, pruned
Dynamic programming	"number of ways", "minimum/maximum X", overlapping subproblems	varies — often $O(n)$ to $O(n^2)$
Bitmask (DP or enumeration)	small n (≤ 20), "subset", "assignment"	$O(2^n)$ or $O(2^n \times n)$
Segment tree / Fenwick Tree	many range queries interleaved with updates	$O(\log n)$ per operation

⚠ **Clue words are strong hints, not proofs**

"Maximize/minimize" suggests greedy OR dynamic programming — the two are often confused. The distinguishing question: can you prove a locally optimal choice never hurts the final answer (greedy), or does the optimal choice at each step genuinely depend on information from multiple possible earlier states (DP)? When unsure, DP is the safer default — it's harder to prove incorrect.

Practice — Chapter 5

Problem 5.1 — Blind pattern matching [Level 2]

Without being told the topic, read this problem and name the pattern:
 "Given a string, find the length of the longest substring without repeating characters."

Hint 1: "substring" (contiguous) + a condition that can grow or shrink as you scan — matches one specific row of the catalog table exactly.

Approach: Sliding window (variable size): expand the window while characters stay unique, shrink from the left when a duplicate is found.

Complexity: $O(n)$.

Chapter 6 — Worked Walkthrough: From Statement to Solution

6.1 The problem

Problem statement

You're given an array of n ($n \leq 2 \times 10^5$) integers and an integer k . Find the length of the longest contiguous subarray whose sum is divisible by k .

6.2 Walking the recognition process from Chapter 1

Step	Applied to this problem
1. Understand	input: array + k . output: a LENGTH (not the subarray itself, not a count).
2. Constraints	$n \leq 2 \times 10^5 \rightarrow$ need $O(n)$ or $O(n \log n)$ (Chapter 2's table).
3. Examples	manually sum a couple of candidate subarrays and check divisibility by hand.
4. Brute force	try every (l, r) pair, sum each subarray, check $\%k == 0$. $O(n^2)$ or $O(n^3)$ naively.
5. Bottleneck	recomputing the sum of every subarray from scratch is the slow part.
6. Pattern match	"subarray sum" $\rightarrow$ prefix sums (Chapter 5's catalog, Part III).
7. Verify	prefix sums give $O(n)$ sums; the question becomes how to use them for the divisibility condition in $O(n)$ too.

6.3 The key observation

Let $\text{prefix}[i]$ be the sum of the first i elements. Subarray $(l, r]$ has sum $\text{prefix}[r] - \text{prefix}[l]$. That sum is divisible by k exactly when $\text{prefix}[r]$ and $\text{prefix}[l]$ have the SAME remainder mod k — because $(\text{prefix}[r] - \text{prefix}[l]) \% k == 0$ iff $\text{prefix}[r] \% k == \text{prefix}[l] \% k$.

✓ Restated goal

Find the largest $r - l$ where $\text{prefix}[r] \% k == \text{prefix}[l] \% k$. This turns the problem into: for each remainder value, remember the EARLIEST index that produced it, and check every later index against that memory.

```
int longestSubarrayDivByK(vector<int>& a, int k) {
    int n = a.size();
    vector<int> firstSeen(k, -2); // -2 = "not seen yet"; -1
    reserved for remainder 0 at index -1
    firstSeen[0] = -1;           // empty prefix (before
    index 0) has remainder 0
    int prefixSum = 0, best = 0;
    for (int r = 0; r < n; r++) {
        prefixSum += a[r];
        int rem = ((prefixSum % k) + k) % k; // Part I's
        negative-modulo fix
        if (firstSeen[rem] == -2) firstSeen[rem] = r; //
        remember first occurrence
        else best = max(best, r - firstSeen[rem]);    //
        reuse the earliest match
    }
    return best;
}
```

6.4 Why storing the FIRST occurrence, not the latest, is correct

✓ The one-line justification

The answer wants the LONGEST subarray, so for a given remainder, pairing the current index with the EARLIEST index sharing that remainder maximizes $r - l$. Overwriting `firstSeen[rem]` with a later index would only ever shrink future candidate lengths.

6.5 Final complexity check against Chapter 2's table

Quantity	Value
Time	$O(n)$ — one pass, $O(1)$ work per element
Space	$O(k)$ for the firstSeen array
Matches $n \leq 2 \times 10^5$'s requirement of $O(n)$ or $O(n \log n)$?	Yes — confirmed in step 7

Quick Revision

- The full recognition process (Chapter 1) applied end to end: understand, constraints, brute force, bottleneck, pattern match, verify.
- "Subarray sum" clue words led to prefix sums (Part III); the divisibility twist turned it into a remainder-matching problem, not a new technique.
- Always circle back to Chapter 2's constraint table at the end to CONFIRM the chosen approach's complexity actually fits — not just at the start.

Part 21 — Master Revision

The Seven-Step Process (Chapter 1, restated)

Step	One-line reminder
1. Underst and	Know exactly what's given and what's asked.
2. Constrai nts	n and the time limit set your TARGET complexity.
3. Examples	Sanity-check your understanding by hand.
4. Brute force	Find the obviously-correct slow solution first.
5. Bottleneck	Identify exactly what makes it slow.
6. Pattern match	Match the bottleneck's shape to a known technique.
7. Verify	Confirm the new approach's complexity actually fits step 2.

Must Remember

Quick Revision

- Recognizing a pattern is a separate, trainable skill from knowing how a technique works internally — both are needed.
- Constraints tell you the target complexity before you've written a line of code; use them to rule out wrong approaches immediately.
- Clue words (Chapter 5) are strong hints, not proof — especially greedy-vs-DP, which needs an actual correctness argument, not just vocabulary matching.
- The bottleneck in your brute force, not the problem's surface description, is usually the most reliable guide to which technique fixes it.
- Always circle back and verify the chosen technique's complexity against the original constraints before committing to implement it.

Common Mistakes Recap

Mistake	Fix
Starting to code before reading constraints	Constraints determine the target complexity — read them FIRST, every time
Pattern-matching on vocabulary alone ("maximize" = greedy)	Verify the exchange-argument / optimal-substructure question before trusting a clue word
Confusing which given number drives complexity (size vs. value range)	Ask specifically: what grows if n grows, versus what grows if the VALUES grow?
Skipping the brute force step entirely	Even when you're confident of the final technique, a brute force catches misunderstandings AND often reveals the bottleneck that motivates the optimization
Forgetting to re-check complexity after designing the optimized approach	Step 7 exists specifically to catch a technique that's elegant but still too slow for the given constraints

Interview-Style Questions

- Why is reading constraints before choosing an algorithm considered one of the two most important habits in competitive programming?
- Explain the difference between pattern-matching on a problem's vocabulary versus pattern-matching on its brute force's bottleneck, and why the second is more reliable.
- Why can greedy and dynamic programming both plausibly apply to a "maximize/minimize" problem, and what question distinguishes which one is actually correct?
- Walk through how you would decide, from constraints alone, whether an $O(n^2)$ solution is likely to be fast enough for $n = 5,000$ with a 2-second time limit.

Cheat Sheet — Part 21

```
// The seven-step process
// 1. Understand exactly what's given and asked
// 2. Constraints -> target complexity (see table below)
// 3. Sanity-check with the given examples by hand
// 4. Find the brute force
// 5. Identify the bottleneck
// 6. Match the bottleneck's shape to a known pattern
// 7. Verify the new approach's complexity against step 2

// Constraints -> complexity
// n <= 10          -> O(n!) / exhaustive search
// n <= 20          -> O(2^n) bitmask
// n <= 40          -> O(2^(n/2)) meet in the middle
// n <= 100         -> O(n^3)
// n <= 1,000       -> O(n^2)
// n <= 100,000     -> O(n log n)
// n <= 1,000,000   -> O(n) or O(n log n)
```

```
// Clue word -> pattern (abbreviated -- see Chapter 5 for the
full catalog)
// "subarray sum"                -> prefix sums
// "contiguous, size/condition" -> sliding window
// "next/prev greater/smaller"  -> monotonic stack
// "same group / connected"     -> DSU
// "must come before"           -> topological sort
// "k-th largest/smallest"      -> heap
// "number of ways" / "min/max" -> dynamic programming (verify,
don't assume)
// small n (<=20), "subset"     -> bitmask / bitmask DP
// many range queries + updates -> segment tree / Fenwick tree
```

Coming up next

With pattern recognition established, the roadmap continues toward the specifically competitive side of things — Codeforces preparation (rating-by-rating skill expectations) and LeetCode preparation (Easy/Medium/Hard pattern-based practice) are next, followed by interview preparation and mock contests. Say which one you'd like built next and under what part number.

